

Base de donnees SQLite

Introduction

Une base de données SQLite est une bonne solution de stockage lorsque vous disposez de données structurées que vous devez stocker de manière persistante et auxquelles vous devez accéder, rechercher et modifier fréquemment.

Lorsque vous utilisez une base de données SQLite, toutes les interactions avec la base de données se font via une instance de la [SQLiteOpenHelper](#) classe qui exécute vos requêtes et gère votre base de données pour vous.

Dans cette pratique, vous créerez une base de données SQLite pour un ensemble de données, afficherez les données récupérées dans un RecyclerView, ajouterez des fonctionnalités pour ajouter, supprimer et modifier les données RecyclerView et les stockerez dans la base de données.

Remarque : Une base de données qui stocke vos données de manière persistante et les synthétise dans un modèle de données est suffisante pour les petites applications Android peu complexes. Dans les chapitres suivants, vous apprendrez à architecturer votre application à l'aide de chargeurs et de fournisseurs de contenu pour mieux séparer les données de l'interface utilisateur. Ces classes vous aideront à déplacer le travail hors du thread d'interface utilisateur afin de rendre l'expérience utilisateur aussi fluide et naturelle que possible. En plus d'améliorer l'expérience utilisateur en supprimant un problème de performance potentiel, elles optimisent votre capacité à étendre et à maintenir votre application.

Important : Dans cet exercice pratique, SQLiteOpenHelper exécute les opérations de base de données dans le thread principal. Dans une application de production, où les opérations de base de données peuvent prendre un certain temps, vous les exécuterez dans un thread d'arrière-plan, par exemple à l'aide d'un [chargeur](#) tel qu'AsyncTaskLoader et CursorLoader.

Ce que vous devez déjà savoir

Vous devriez être familier avec:

- Création, création et exécution d'applications dans Android Studio.
- Affichage des données dans un RecyclerView.
- Utilisation d'adaptateurs comme intermédiaires entre les données et les vues.
- Ajout de gestionnaires d'événements onClick aux vues et création dynamique de gestionnaires onClick.
- Démarrer une deuxième activité et renvoyer des données à partir de celle-ci.
- Transmission de données entre activités à l'aide d'extras d'intention.
- Utilisation d'une vue EditText pour obtenir les données saisies par l'utilisateur.

Vous avez également besoin d'une compréhension de base des bases de données SQL, de la façon dont elles sont organisées en tables de lignes et de colonnes, et de la

Langage SQL. Voir le guide SQLite

Ce que vous apprendrez

- Créez et gérez une base de données SQLite avec un SQLiteOpenHelper.
- Implémentez les fonctionnalités d'insertion, de suppression, de mise à jour et de requête via votre assistant ouvert.
- Utilisez un adaptateur et un gestionnaire de clics personnalisé pour permettre aux utilisateurs d'interagir avec la base de données à partir de l'interface utilisateur.

Ce que tu feras

Vous commencez avec une application qui est la même que l'application de liste de mots RecyclerView que vous avez créée précédemment, avec des droits d'utilisateur supplémentaires.

éléments d'interface déjà ajoutés pour vous, afin que vous puissiez vous concentrer sur le code de la base de données.

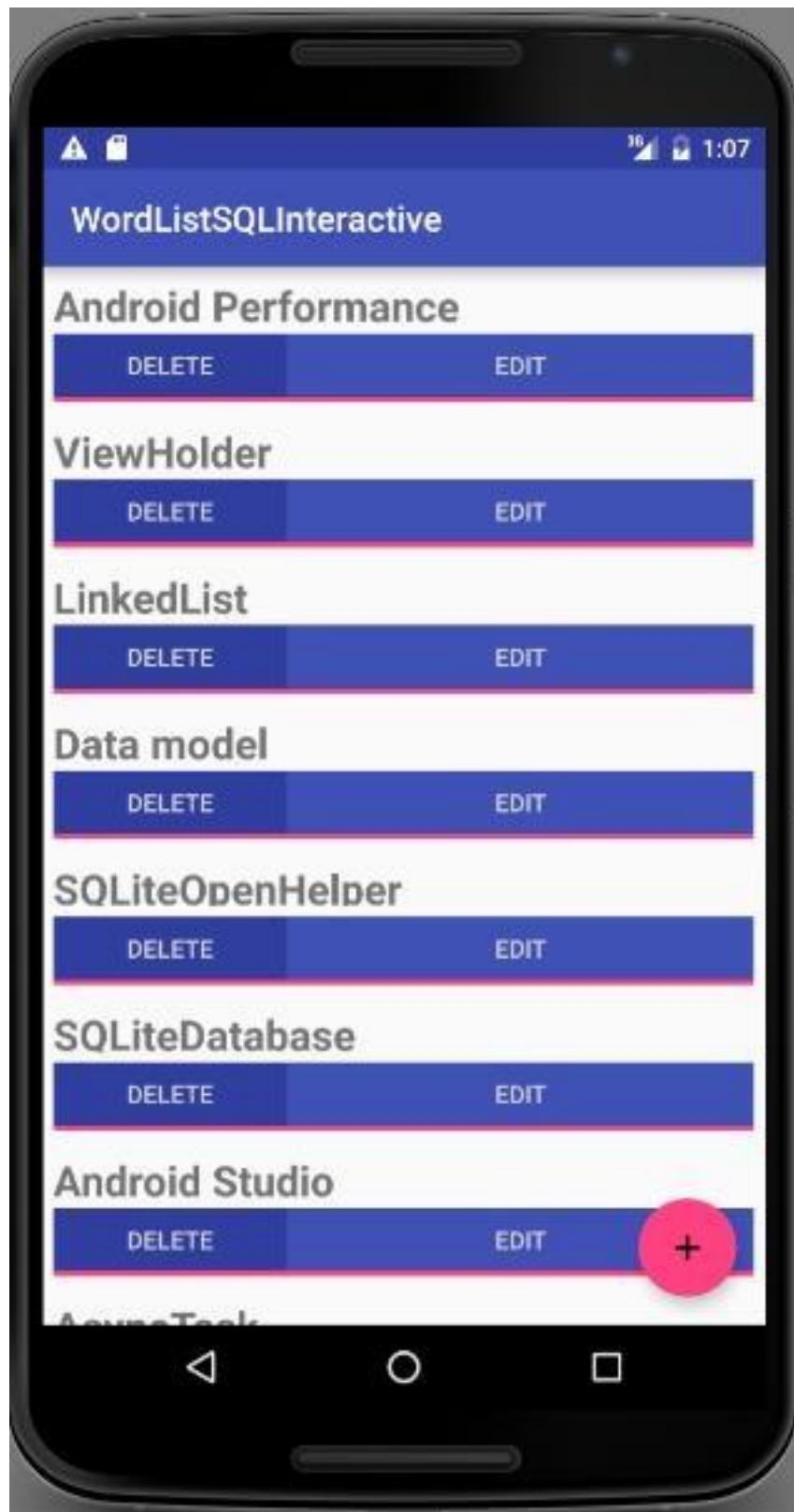
Vous étendrez et modifierez l'application de base pour :

- Implémentez une classe personnalisée pour modéliser vos données.
- Créez une sous-classe de SQLiteOpenHelper qui crée et gère la base de données de votre application.
- Afficher les données de la base de données dans RecyclerView.
- Implémentez des fonctionnalités pour ajouter, modifier et supprimer des données dans l'interface utilisateur et stocker les modifications dans la base de données.

2. Aperçu de l'application

À partir d'une application squelette, vous ajouterez des fonctionnalités à :

- Afficher les mots d'une base de données SQLite dans un RecyclerView.
- Chaque mot peut être modifié ou supprimé.
- Vous pouvez ajouter de nouveaux mots et les stocker dans la base de données.



La version minimale du SDK est API15 : Android 4.0.3 IceCreamSandwich et le SDK *cible* est la version actuelle d'Android.

3. Tâche 0 : Téléchargez et exécutez le code de démarrage

Afin de vous épargner du travail, en particulier l'écriture d'activités non liées à la base de données et de code d'interface utilisateur, vous devez obtenir le code de démarrage pour cet exercice pratique.

1. Téléchargez le code de démarrage [WordListSqlStarterCode](#) .
2. Ouvrez l'application dans Android Studio.
3. Exécutez l'application. L'interface utilisateur devrait s'afficher comme dans la capture d'écran précédente. Tous les mots affichés doivent être des « placeholder ». Cliquer sur les boutons ne fait rien.

4. Tâche 1 : Étendre SQLiteOpenHelper pour créer et remplir une base de données

Les applications Android peuvent utiliser des bases de données SQLite standard pour stocker des données. Cet exercice pratique n'enseigne pas SQLite, mais montre comment l'utiliser dans une application Android.

[SQLiteOpenHelper](#) Il s'agit d'une classe utilitaire du SDK Android permettant d'interagir avec un objet [de base de données SQLite](#) . Elle inclut `onCreate()`des `onUpdate()`méthodes à implémenter, ainsi que des méthodes pratiques d'insertion, de suppression, de mise à jour et d'interrogation pour toutes vos interactions avec la base de données.

La `SQLiteOpenHelper` classe se charge d'ouvrir la base de données si elle existe, de la créer si elle n'existe pas et de la mettre à niveau si nécessaire.

Remarque : Vous pouvez avoir plusieurs bases de données par application, et plusieurs assistants ouverts pour les gérer. Cependant, pour des raisons de performances et de simplicité architecturale, pensez à créer plusieurs tables dans la même base de données.

1.1 Créer une classe squelette WordListOpenHelper

La première étape de l'ajout d'une base de données à votre code consiste toujours à créer une sous-classe `SQLiteOpenHelper` et à implémenter ses méthodes.

1. Créez une nouvelle classe Java **WordListOpenHelper** avec la signature suivante.

```
public class WordListOpenHelper extends SQLiteOpenHelper {}
```

2. Dans l'éditeur de code, survolez l'erreur, puis cliquez sur l'image de l'ampoule et sélectionnez « **Implémenter les méthodes** » . Assurez-vous que les deux méthodes sont sélectionnées et cliquez sur « **OK** » .
3. Ajoutez le constructeur manquant pour `WordListOpenHelper`. (Vous définirez ensuite les constantes non définies.)

```
public WordListOpenHelper(Context context) {  
    super(context, DATABASE_NAME, null, DATABASE_VERSION);  
}
```

1.2 Ajouter des constantes de base de données à WordListOpenHelper

1. En haut de la WordListOpenHelper classe, définissez les constantes des tables, des lignes et des colonnes comme indiqué dans le code ci-dessous. Cela devrait éliminer toutes les erreurs.

```
// It's a good idea to always define a log tag like this.
private static final String TAG = WordListOpenHelper.class.getSimpleName();

// has to be 1 first time or app will crash
private static final int DATABASE_VERSION = 1;
private static final String WORD_LIST_TABLE = "word_entries";
private static final String DATABASE_NAME = "wordlist";

// Column names...
public static final String KEY_ID = "_id";
public static final String KEY_WORD = "word";

// ... and a string array of columns.
private static final String[] COLUMNS = { KEY_ID, KEY_WORD };
```

2. Exécutez votre code pour vous assurer qu'il ne contient plus d'erreurs.

1.3 Construisez la requête SQL et le code pour créer la base de données

Les requêtes SQL peuvent devenir assez complexes. Il est recommandé de les construire séparément du code qui les utilise. Cela améliore la lisibilité du code et facilite le débogage.

Continuer à ajouter du code à WordListOpenHelper.java :

1. Sous les constantes, ajoutez le code suivant pour construire la requête. Consultez le guide SQLite pour comprendre cette requête.

```
// Build the SQL query that creates the table.
private static final String WORD_LIST_TABLE_CREATE =
    "CREATE TABLE " + WORD_LIST_TABLE + " (" +
        KEY_ID + " INTEGER PRIMARY KEY, " +
        // id will auto-increment if no value passed
        KEY_WORD + " TEXT );";
```

2. Ajoutez des variables d'instance pour les références aux bases de données accessibles en écriture et en lecture. Le stockage de ces références vous évite d'avoir à obtenir une référence de base de données à chaque lecture ou écriture.

```
private SQLiteDatabase mWritableDatabase;
private SQLiteDatabase mReadableDB;
```

3. Dans la onCreate méthode, ajoutez du code pour créer une base de données et la table (la classe d'assistance ne crée pas une autre base de données, si elle existe déjà.)

```
@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(WORD_LIST_TABLE_CREATE);
}
```

4. Corrigez l'erreur en renommant l'argument de la méthode de SQLiteDatabase en db.

1.4 Créer la base de données dans onCreate de MainActivity

Pour créer la base de données, créez une instance de la `WordListOpenHelper` classe que vous venez d'écrire.

1. Ouvrez **MainActivity.java** et ajoutez une variable d'instance pour l'assistant d'ouverture :

```
private WordListOpenHelper mDB;
```

2. Dans `onCreate`, initialisez `mDB` avec une instance de `WordListOpenHelper`. Ceci appelle `onCreate`, `WordListOpenHelper` crée la base de données.

```
mDB = new WordListOpenHelper(this);
```

3. Ajoutez un point d'arrêt, exécutez l'application avec le débogueur et vérifiez qu'il `mDB` s'agit d'une instance pour `WordListOpenHelper`.

1.5 Ajouter des données à la base de données

La liste de mots de votre application peut provenir de nombreuses sources. Elle peut être entièrement créée par l'utilisateur, téléchargée sur Internet ou générée à partir d'un fichier intégré à votre APK. Pour cela, vous allez alimenter votre base de données avec une petite quantité de données codées en dur.

Notez que l'acquisition, la création et le formatage des données constituent un sujet entièrement distinct qui n'est pas abordé dans ce cours.

1. Ouvrez **WordListOpenHelper.java**.
2. Dans `onCreate`, après avoir créé la base de données, ajoutez un appel de fonction à

```
fillDatabaseWithData(db);
```

Ensuite, implémentez la `fillDatabaseWithData()` méthode dans `WordListOpenHelper`.

3. Implémenter la méthode stub.

```
private void fillDatabaseWithData(SQLiteDatabase db){}
```

4. À l'intérieur de la méthode, déclarez une chaîne de mots comme données fictives.

```
String[] words = {"Android", "Adapter", "ListView", "AsyncTask",  
"Android Studio", "SQLiteDatabase", "SQLOpenHelper",  
"Data model", "ViewHolder", "Android Performance",  
"OnClickListener"};
```

5. Créez un conteneur pour les données. La méthode d'insertion que vous appellerez ensuite requiert les valeurs pour remplir une ligne en tant qu'instance de [ContentValues](#). Une instance `ContentValues` stocke les données d'une ligne sous forme de paires clé-valeur, où la clé est le nom de la colonne et la valeur est la valeur à définir.

```
// Create a container for the data.
```

```
ContentValues values = new ContentValues();
```

6. Ajoutez la clé/valeur de la première ligne à la section « valeurs », puis insérez cette ligne dans la base de données. Répétez l'opération pour tous les mots de votre tableau.
 - `db.insert` est une méthode pratique de [SQLiteDatabase](#) permettant d'insérer une ligne dans la base de données. (Cette méthode est pratique, car vous n'avez pas besoin d'écrire la requête SQL vous-même.)
 - Le premier argument de `db.insert` est le nom de la table, `WORD_LIST_TABLE`.
 - Le deuxième argument est un `String` `nullColumnHack`. Il s'agit d'une solution SQL permettant d'insérer des lignes vides. Consultez [la documentation de insert\(\)](#). Utilisez `null` pour cet argument.

- Le troisième argument doit être un conteneur [ContentValues](#) contenant les valeurs nécessaires pour remplir la ligne. Cet exemple ne comporte qu'une seule colonne, « words », comme représenté par la constante KEY_WORD définie précédemment ; pour les tables à plusieurs colonnes, ajoutez les valeurs de chaque colonne à ce conteneur.

```
for (int i=0; i < words.length; i++) {
    // Put column/value pairs into the container.
    // put() overrides existing values.
    values.put(KEY_WORD, words[i]);
    db.insert(WORD_LIST_TABLE, null, values);
}
```

7. Avant d'exécuter et de tester votre application, vous devez effacer les données de votre base de données SQLite et la supprimer. Nous pouvons ensuite exécuter notre application et la recréer afin que la base de données soit initialisée avec les données de départ. Vous pouvez désinstaller l'application de votre appareil ou effacer toutes ses données depuis **Paramètres > Applications > Liste de mots > Stockage > Effacer les données** sur votre émulateur Android ou votre appareil physique .
 8. Exécutez votre application. Vous ne constaterez aucun changement dans l'interface utilisateur.
- Vérifiez les journaux et assurez-vous qu'il n'y a aucune erreur avant de continuer. Si vous rencontrez des erreurs, lisez attentivement les messages logcat et utilisez des ressources, comme Stack Overflow, si vous êtes bloqué.
 - Vous pouvez également vérifier dans les paramètres que le stockage des utilisateurs de l'application.

5. Tâche 2 : Créer un modèle de données pour un seul mot

Un modèle de données est une classe qui encapsule une structure de données complexe et fournit une API permettant d'accéder aux données de cette structure et de les manipuler. Un modèle de données est nécessaire pour transmettre les données extraites de la base de données à l'interface utilisateur.

Pour cette application pratique, le modèle de données contient uniquement le mot et son identifiant. Bien que l'identifiant unique soit généré par la base de données, il est nécessaire de le transmettre à l'interface utilisateur. Cela permettra d'identifier le mot modifié par l'utilisateur.

2.1 Créez un modèle de données pour vos données de mots

1. Créez une nouvelle classe et appelez-la `WordItem`.
2. Ajoutez les variables de classe suivantes.

```
private int mId;
private String mWord;
```

3. Ajoutez un constructeur vide.
4. Ajoutez des getters et des setters pour l'identifiant et le mot.
5. Exécutez votre application. Vous ne verrez aucun changement d'interface utilisateur, mais aucune erreur ne devrait se produire.

Solution:

```
public class WordItem {
    private int mId;
```

```

private String mWord;

public WordItem() {}

public int getId() {return this.mId;}

public String getWord() {return this.mWord;}

public void setId(int id) {this.mId = id;}

public void setWord(String word) {this.mWord = word;}
}

```

6. Tâche 3 : Implémenter la méthode query() dans WordListOpenHelper

La `query()` méthode récupère les lignes de la base de données sélectionnées par une requête SQL.

Pour cet exemple, afin d'afficher les mots dans la RecyclerView, nous devons les extraire de la base de données, un par un, selon les besoins. Le mot recherché est identifié par sa position dans la vue.

En tant que telle, la méthode de requête possède un paramètre pour la position demandée et renvoie un `WordItem`.

3.1. Implémenter la méthode query()

1. Créez une méthode de requête qui prend un argument de position entier et renvoie un `WordItem`.

```

public WordItem query(int position) {
}

```

2. Construisez une requête qui renvoie uniquement la nième ligne du résultat. Utilisez `LIMIT`-la avec la position comme ligne et 1 comme nombre de lignes.

```

String query = "SELECT * FROM " + WORD_LIST_TABLE +
    " ORDER BY " + KEY_WORD + " ASC " +
    "LIMIT " + position + ",1";

```

3. Instanciez une `Cursor` variable pour contenir le résultat de la base de données.

```

Cursor cursor = null;

```

Il `SQLiteDatabase` présente toujours les résultats sous `Cursor` forme de tableau qui ressemble à une base de données SQL.

Un curseur est un pointeur pointant vers une ligne de données structurées. Il peut être considéré comme un tableau de lignes. La `Cursor` classe fournit des méthodes pour déplacer le curseur dans cette structure et des méthodes pour extraire les données des colonnes de chaque ligne.

4. Instancier une `WordItem` entrée.

```

WordItem entry = new WordItem();

```

5. Ajoutez un bloc try/catch/finally.


```
try {} catch (Exception e) {} finally {}
```

6. À l'intérieur du bloc try,

1. obtenir une base de données lisible si elle n'existe pas.

```
if (mReadableDB == null) {  
    mReadableDB = getReadableDatabase();  
}
```

2. envoyer une requête brute à la base de données et stocker le résultat dans un curseur.

```
cursor = mReadableDB.rawQuery(query, null);
```

La méthode d'aide à la requête ouverte peut construire une chaîne de requête SQL et l'envoyer sous forme de requête brute à la base de données, qui renvoie un curseur. Si vos données sont fournies par votre application et sous votre contrôle total, vous pouvez utiliser la méthode raw query().

1. Déplacez le curseur sur le premier élément.

```
cursor.moveToFirst();
```

2. Définissez l'ID et le mot de l' `WordItem` entrée sur les valeurs renvoyées par le curseur.

```
entry.setId(cursor.getInt(cursor.getColumnIndex(KEY_ID)));  
entry.setWord(cursor.getString(cursor.getColumnIndex(KEY_WORD)));
```

7. Dans le bloc catch, enregistrez l'exception.

```
Log.d(TAG, "EXCEPTION! " + e);
```

8. Dans le bloc finally, fermez le curseur et renvoyez l'entrée `WordItem`.

```
cursor.close();  
return entry;
```

Solution:

```
public WordItem query(int position) {  
    String query = "SELECT * FROM " + WORD_LIST_TABLE +  
        " ORDER BY " + KEY_WORD + " ASC " +  
        "LIMIT " + position + ",1";  
  
    Cursor cursor = null;  
    WordItem entry = new WordItem();  
  
    try {  
        if (mReadableDB == null) {  
            mReadableDB = getReadableDatabase();  
        }  
        cursor = mReadableDB.rawQuery(query, null);  
        cursor.moveToFirst();  
        entry.setId(cursor.getInt(cursor.getColumnIndex(KEY_ID)));  
        entry.setWord(cursor.getString(cursor.getColumnIndex(KEY_WORD)));  
    } catch (Exception e) {  
        Log.d(TAG, "QUERY EXCEPTION! " + e.getMessage());  
    } finally {  
        cursor.close();  
        return entry;  
    }  
}
```

3.2. La méthode onUpgrade

Chaque utilisateur `SQLiteOpenHelper` doit implémenter la `onUpgrade()` méthode qui détermine ce qui se passe en cas de changement de version de la base de données. Cela peut se produire si des utilisateurs de votre application utilisent une ancienne version de la base de données. Cette méthode est déclenchée à la première ouverture d'une base de données. L'action par défaut habituelle consiste à supprimer la base de données actuelle et à la recréer.

Important : bien qu'il soit acceptable de supprimer la table dans un exemple d'application, dans une application de production, vous devez migrer soigneusement les précieuses données de l'utilisateur.

Vous pouvez utiliser le code ci-dessous pour implémenter la `onUpgrade()` méthode pour cet exemple.

Code standard pour `onUpgrade()` :

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    Log.w(WordListOpenHelper.class.getName(),
        "Upgrading database from version " + oldVersion + " to "
        + newVersion + ", which will destroy all old data");

    db.execSQL("DROP TABLE IF EXISTS " + WORD_LIST_TABLE);
    onCreate(db);
}
```

7. Tâche 4 : Afficher les données dans RecyclerView

Vous disposez désormais d'une base de données contenant des données. Vous allez maintenant mettre à jour `WordListAdapter` et `MainActivity` pour récupérer et afficher ces données.

4.1. Mettre à jour `WordListAdapter` pour afficher les `WordItems`

1. Ouvrez **`WordListAdapter`**.
2. Remplacez `onBindViewHolder` le code affichant les données fictives par du code permettant d'extraire un élément de la base de données et de l'afficher. Vous remarquerez une erreur sur `mDB`.

```
WordItem current = mDB.query(position);
holder.wordItemView.setText(current.getWord());
```

3. Déclarer `mDB` comme une variable d'instance.

```
WordListOpenHelper mDB;
```

4. Pour obtenir la valeur de `mDB`, modifiez le constructeur pour `WordListAdapter` et ajoutez un deuxième paramètre pour le `WordListOpenHelper`.
5. Affectez la valeur du paramètre à `mDB`. Votre constructeur devrait ressembler à ceci :

```
public WordListAdapter(Context context, WordListOpenHelper db) {
    mInflater = LayoutInflater.from(context);
    mContext = context;
```

```
mDB = db;
}
```

Cela génère une erreur dans `MainActivity`, car vous avez ajouté un argument au `WordListAdapter` constructeur.

6. Ouvrez **MainActivity** et ajoutez l' `mDB` argument manquant.

```
mAdapter = new WordListAdapter (this, mDB);
```

7. Exécutez votre application. Vous devriez voir tous les mots de la base de données.

8. Tâche 5 : Ajouter de nouveaux mots à la base de données

Lorsque l'utilisateur clique sur le FAB, une activité s'ouvre qui lui permet de saisir un mot qui sera ajouté à la base de données lorsqu'il cliquera sur Enregistrer.

Le code de démarrage vous fournit l'écouteur de clics et le `EditWordActivity` démarrage en cliquant sur le FAB. Vous ajouterez le code spécifique à la base de données et relierez les éléments ensemble, de bas en haut, comme vous venez de le faire avec la méthode de requête.

5.1. Écrire la méthode insert()

Dans `WordListOpenHelper`:

1. Créez la `insert()` méthode avec la signature suivante. L'utilisateur fournit un mot et la méthode renvoie l'identifiant de la nouvelle entrée. Les identifiants générés peuvent être grands ; `insert` renvoie donc un nombre de type long.

```
public long insert(String word){}
```

2. Déclarez une variable pour l'ID. Si l'opération d'insertion échoue, la méthode renvoie 0.

```
long newId = 0;
```

3. Comme précédemment, créez une `ContentValues` valeur pour les données de ligne.

```
ContentValues values = new ContentValues();
values.put(KEY_WORD, word);
```

4. Placez votre opération de base de données dans un bloc try/catch.

```
try {} catch (Exception e) {}
```

5. Obtenez une base de données accessible en écriture si elle n'existe pas déjà.

```
if (mWritableDatabase == null) {
    mWritableDatabase = getWritableDatabase();
}
```

6. Insérer la ligne.

```
newId = mWritableDatabase.insert(WORD_LIST_TABLE, null, values);
```

7. Enregistrez l'exception.

```
Log.d(TAG, "INSERT EXCEPTION! " + e.getMessage());
```

8. Renvoyer l'identifiant.

```
return newId;
```

Solution:

```
public long insert(String word){
    long newId = 0;
    ContentValues values = new ContentValues();
    values.put(KEY_WORD, word);
    try {
        if (mWritableDatabase == null) {
            mWritableDatabase = getWritableDatabase();
        }
        newId = mWritableDatabase.insert(WORD_LIST_TABLE, null, values);
    } catch (Exception e) {
        Log.d(TAG, "INSERT EXCEPTION! " + e.getMessage());
    }
    return newId;
}
```

5.2. Récupérer le mot à insérer de l'utilisateur et mettre à jour la base de données

Le code de démarrage est fourni avec une méthode `EditWordActivity` qui récupère un mot de l'utilisateur et le renvoie à l'activité principale. Dans cette `MainActivity` méthode, il suffit de renseigner la `onActivityResult` méthode.

1. Vérifiez que le résultat provient de l'activité correcte et récupérez le mot que l'utilisateur a saisi à partir des extras.

```
if (requestCode == WORD_EDIT) {
    if (resultCode == RESULT_OK) {
        String word = data.getStringExtra(EditWordActivity.EXTRA_REPLY);
```

2. Si le mot n'est pas vide, vérifiez si un identifiant avec les extras nous a été transmis. S'il n'y en a pas, insérez un nouveau mot. Dans la tâche suivante, vous mettrez à jour le mot existant si un identifiant est transmis.

```
if (!TextUtils.isEmpty(word)) {
    int id = data.getIntExtra(WordListAdapter.EXTRA_ID, -99);
    if (id == WORD_ADD) {
        mDB.insert(word);
    }
}
```

3. Pour mettre à jour l'interface utilisateur, informez l'adaptateur que les données sous-jacentes ont changé.

```
mAdapter.notifyDataSetChanged();
```

4. Si le mot est vide parce que l'utilisateur n'a rien saisi, affichez un message pour l'en informer. N'oubliez pas de fermer toutes les parenthèses.

```
} else {
    Toast.makeText(
        getApplicationContext(),
        R.string.empty_not_saved,
        Toast.LENGTH_LONG).show();
    }
}
```

Solution:

```
if (requestCode == WORD_EDIT) {
    if (resultCode == RESULT_OK) {
        String word = data.getStringExtra(EditWordActivity.EXTRA_REPLY);
        // Update the database
        if (!TextUtils.isEmpty(word)) {
            int id = data.getIntExtra(WordListAdapter.EXTRA_ID, -99);
            if (id == WORD_ADD) {
                mDB.insert(word);
            }
            // Update the UI
            mAdapter.notifyDataSetChanged();
        } else {
            Toast.makeText(
                getApplicationContext(),
                R.string.empty_not_saved,
                Toast.LENGTH_LONG).show();
        }
    }
}
```

5.3. Implémenter getItemCount()

Pour que les nouveaux éléments s'affichent correctement, `getItemCount` de `WordListAdapter` doit renvoyer le nombre réel d'entrées dans la base de données au lieu du nombre de mots dans la liste de départ des mots.

1. Modifiez `getItemCount` le code ci-dessous, ce qui déclenchera une erreur.

```
return (int) mDB.count();
```

2. Ouvrir `WordListOpenHelper` et implémenter `count()` pour renvoyer le nombre d'entrées dans la base de données.

```
public long count() {
    if (mReadableDB == null) {
        mReadableDB = getReadableDatabase();
    }
    return DatabaseUtils.queryNumEntries(mReadableDB, WORD_LIST_TABLE);
}
```

3. Exécutez votre application et ajoutez quelques mots.

9. Tâche 6 : Supprimer des mots de la base de données

Pour implémenter la fonctionnalité de suppression, vous devez :

- Implémenter la `delete()` méthode dans `WordListOpenHelper`
- Ajoutez un gestionnaire de clic au `DELETE` bouton dans `WordListAdapter`

6.1. Écrire la méthode delete()

Utilisez la `delete()` méthode on `SQLiteDatabase` pour supprimer une entrée de la base de données. Ajoutez la méthode `delete` à `WordListOpenHelper` that :

1. Créez la méthode stub pour `delete()`, qui prend un argument `int` pour l'ID de l'élément à supprimer et renvoie le nombre de lignes supprimées.

```
public int delete(int id) {}
```

2. Déclarez une variable pour contenir le résultat.

```
int deleted = 0;
```

3. Pour l'insertion, ajoutez un bloc `try`.

```
try {} catch (Exception e) {}
```

4. Obtenez une base de données accessible en écriture, si nécessaire.

```
if (mWritableDatabase == null) {  
    mWritableDatabase = getWritableDatabase();  
}
```

5. Appelez `delete` sur `WORD_LIST_TABLE`, en sélectionnant par `KEY_ID` et en passant la valeur de l'ID comme argument. Le «`?`» est un espace réservé qui est rempli avec la chaîne. C'est une méthode plus sécurisée pour créer des requêtes.

```
deleted = mWritableDatabase.delete(WORD_LIST_TABLE,  
    KEY_ID + " = ? ", new String[] {String.valueOf(id)});
```

6. Imprimer un message de journal pour les exceptions.

```
Log.d (TAG, "DELETE EXCEPTION! " + e.getMessage());
```

7. Renvoie le nombre de lignes supprimées.

```
return deleted;
```

Solution:

```
public int delete(int id) {  
    int deleted = 0;  
    try {  
        if (mWritableDatabase == null) {  
            mWritableDatabase = getWritableDatabase();  
        }  
        deleted = mWritableDatabase.delete(WORD_LIST_TABLE, //table name  
            KEY_ID + " = ? ", new String[] {String.valueOf(id)});  
    } catch (Exception e) {  
        Log.d (TAG, "DELETE EXCEPTION! " + e.getMessage());  
    }  
    return deleted;  
}
```

6.2. Ajouter un gestionnaire de clics au bouton SUPPRIMER

Vous pouvez maintenant ajouter un gestionnaire de clic au `DELETE` bouton qui appelle la `delete()` méthode que vous venez d'écrire.

Examinez la `MyButtonOnClickListener` classe dans votre code de démarrage. Elle `MyButtonOnClickListener` implémente un écouteur de clics qui stocke l'identifiant et le mot nécessaire pour modifier la base de données.

Chaque détenteur de vue, lorsqu'il est attaché (lié) à `RecyclerView` la `onBindViewHolder` méthode `WordListAdapter`, doit également attacher un écouteur de clic au `DELETE` bouton, en transmettant l'identifiant et le mot au `MyButtonOnClickListener` constructeur. Ces valeurs sont ensuite utilisées par le `onClick` gestionnaire pour supprimer l'élément correct et notifier l'adaptateur de la suppression.

Notez que vous ne pouvez pas utiliser l'argument de position passé à `onBindViewHolder`, car il risque d'être obsolète au moment de l'appel du gestionnaire de clics. Vous devez conserver une référence au détenteur de la vue et obtenir la position avec `getAdapterPosition()`.

Solution:

```
// Keep a reference to the view holder for the click listener
final WordViewHolder h = holder; // needs to be final for use in callback

// Attach a click listener to the DELETE button.
holder.delete_button.setOnClickListener(
    new MyButtonOnClickListener(current.getId(), null) {

        @Override
        public void onClick(View v) {
            int deleted = mDB.delete(id);
            if (deleted >= 0)
                notifyItemRemoved(h.getAdapterPosition());
        }
    });
```

10. Tâche 7 : Mettre à jour les mots dans la base de données

Pour mettre à jour les mots existants, vous devez :

- Ajoutez une `update()` méthode à `WordListOpenHelper`.
- Ajoutez un gestionnaire de clic au `EDIT` bouton de votre vue.

7.1. Écrire la méthode `update()`

Vous utilisez la `update()` méthode sur `SQLiteDatabase` pour mettre à jour une entrée existante dans la base de données.

1. Ajoutez une méthode à `WordListOpenHelper` cela :
 - Prend un identifiant entier et un `String` mot pour ses arguments et renvoie un entier.

```
public int update(int id, String word)
```

- Initialise `int mNumberOfRowsUpdated` à -1.

```
int mNumberOfRowsUpdated = -1;
```

2. À l'intérieur d'un bloc try, procédez comme suit :
3. Obtenez une base de données SQLiteDatabase accessible en écriture s'il n'y en a pas déjà une.

```
if (mWritableDB == null) {  
    mWritableDB = getWritableDatabase();  
}
```

4. Créez une nouvelle instance de ContentValues au KEY_WORDmot qui lui correspond.

```
ContentValues values = new ContentValues();  
values.put(KEY_WORD, word);
```

5. Appelez db.updateen utilisant les arguments suivants :

```
mNumberOfRowsUpdated = db.update(WORD_LIST_TABLE,  
    values, // new values to insert  
    // selection criteria for row (the _id column)  
    KEY_ID + " = ?",  
    //selection args; value of id  
    new String[]{String.valueOf(id)});
```

6. Dans le bloc catch, imprimez un message de journal si des exceptions sont rencontrées.

```
Log.d(TAG, "UPDATE EXCEPTION: " + e.getMessage());
```

7. Renvoie le nombre de lignes mises à jour, qui doit être -1 (échec), 0 (rien de mis à jour) ou 1 (succès).

```
return mNumberOfRowsUpdated;
```

Solution:

```
public int update(int id, String word) {  
    int mNumberOfRowsUpdated = -1;  
    try {  
        if (mWritableDB == null) {  
            mWritableDB = getWritableDatabase();  
        }  
        ContentValues values = new ContentValues();  
        values.put(KEY_WORD, word);  
        mNumberOfRowsUpdated = mWritableDB.update(WORD_LIST_TABLE,  
            values,  
            KEY_ID + " = ?",  
            new String[]{String.valueOf(id)});  
    } catch (Exception e) {  
        Log.d(TAG, "UPDATE EXCEPTION! " + e.getMessage());  
    }  
    return mNumberOfRowsUpdated;  
}
```

7.2. Ajouter un écouteur de clic au bouton MODIFIER

Voici le code de l'écouteur de clic « Modifier » lorsque nous lions « the » Viewà la onBindViewHolderméthode de WordListAdapter. Cet écouteur n'a rien de spécifique à la base de données. Il démarre l' EditWordActivityactivité à l'aide d'un objet Intentet lui transmet l'identifiant, la position et le mot actuels dans les extras.

Si vous obtenez une erreur sur la `EXTRA_POSITION` constante, ajoutez-la avec une valeur de `"POSITION"`,

Solution:

```
// Attach a click listener to the EDIT button.
holder.edit_button.setOnClickListener(new MyButtonOnClickListener(
    current.getId(), current.getWord()) {

    @Override
    public void onClick(View v) {
        Intent intent = new Intent(mContext, EditWordActivity.class);

        intent.putExtra(EXTRA_ID, id);
        intent.putExtra(EXTRA_POSITION, h.getAdapterPosition());
        intent.putExtra(EXTRA_WORD, word);

        // Start an empty edit activity.
        ((Activity) mContext).startActivityForResult(
            intent, MainActivity.WORD_EDIT);
    }
});
```

7.3. Ajout d'une mise à jour à onActivityResult

Une fois implémenté, cliquer sur « Modifier » lance une activité qui affiche le mot actuel et permet à l'utilisateur de le modifier. Pour effectuer la mise à jour,

1. Ajoutez une ligne de code à la `onActivityResult` méthode dans votre `MainActivity`.

```
else if (id >= 0) {
    mDB.update(id, word);
}
```

2. Exécutez votre application et jouez avec elle

7.4. Considérations relatives à la conception et aux erreurs

- Les méthodes que vous avez écrites pour ajouter, mettre à jour et supprimer des entrées dans la base de données supposent toutes que leurs entrées sont valides. Ceci est acceptable pour un exemple de code, car il vise à vous enseigner les fonctionnalités de base d'une base de données SQLite. Par conséquent, tous les cas limites ne sont pas pris en compte, toutes les valeurs ne sont pas testées et tout le monde est supposé se comporter correctement. S'il s'agissait d'une application de production, les considérations de sécurité seraient plus strictes et le contenu devrait être testé pour vérifier sa validité jusqu'à ce que vous soyez certain qu'il n'est pas malveillant.
- Dans une application de production, vous devez détecter des exceptions spécifiques et les gérer de manière appropriée.
- Vous avez testé le bon fonctionnement de l'application en l'exécutant. Pour une application de production avec des données réelles, des tests plus approfondis seront nécessaires, par exemple à l'aide de tests unitaires et d'interface.
- Pour cet exercice pratique, vous avez créé le schéma et les tables de la base de données à partir de la `SQLiteOpenHelper` classe. Cela suffit pour un exemple simple, comme celui-ci. Pour une application plus complexe, il est préférable de séparer les définitions de schéma du reste du code dans une classe

d'assistance non instanciable. Vous apprendrez comment procéder dans le chapitre sur les fournisseurs de contenu.

- Comme mentionné précédemment, certaines opérations de base de données peuvent être longues et doivent être effectuées en arrière-plan. Utilisez-les `AsyncTask` pour les opérations longues. Utilisez des chargeurs pour charger de grandes quantités de données.

11. Résumé

Dans ce chapitre, vous avez appris à

- Utilisez a `SQLiteDatabase` pour stocker les données utilisateur de manière persistante.
- Gérez votre base de données avec un outil `SQLiteOpenHelper`. Récupérez et affichez les données de la base.
- Modifiez les données dans l'interface utilisateur et répercutez ces modifications dans la base de données